

PyQCU CUDA-C++ Clover MultiGrid 求解器：Schur 一致多重网格的调试与性能优化报告

ZhangXin

IMP, CAS

2026-08-02

摘要

针对 logs/clover_multigrid.log 中 C++ CUDA MultiGrid 求解器相比 BiStabCG 性能优势不明显的问题, 本文系统地调试并优化了 cpp/cuda/qcu 中的多线程、多精度 Clover MultiGrid 求解器。根因是: level-0 采用奇偶预条件 (Schur 补) BiStabCG 求解, 但原有的粗空间基于全算子 D 的零模构建, 与 Schur 算子 $S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}$ 的低模不匹配, 导致 V-cycle 修正失效甚至使残差反弹。我们将粗空间改为基于 S 自身零模构建, 并将粗算子 $A_c = P^\top S P$ 物化为 33 张量 stencil (on-site + 最近邻 + 对角耦合), 实现了与 pyqcu/solver/_multigrid.py (support_parity=True) 一致的 Schur 一致多重网格。最终求解器正确 (与 BiStabCG 相对差 $\sim 4 \times 10^{-7}$), 主求解算法迭代次数从 BiStabCG 的 ~ 90 次降至 ~ 63 -68 次 (减少约 30%)。

目录

1 背景与目标	2
2 根因诊断	2
2.1 全算子粗空间对 Schur 解无效	2
2.2 Schur 一致的粗空间有效	2
2.3 粗算子的对角耦合	3
3 算法设计	3
3.1 level-0 (最细层)	3
3.2 粗层	3
3.3 零模的生成与正确性	4
4 修复的 Bug	4
5 正确性验证	4
6 性能分析	5
6.1 迭代次数	5
6.2 墙钟时间与硬件限制	5
6.3 墙钟时间 (8×16×16×16, 本机 GPU 210 MHz 启动受限)	5
6.4 参数扫描	5

1 背景与目标

PyQCU 的 CUDA-C++ Clover MultiGrid 求解器(`applyCloverMultigridQcu`, 实现于 `cpp/cuda/qcu/include`) 目标是:

- 最终功能与 `applyCloverBistabCgDslashQcu` (奇偶预条件 Clover dslash) 一致, 即求解奇偶预条件 (Schur 补) 系统 $Sx_o = b_o$;
- 主求解算法迭代次数比 BiStabCG 少, 耗时更短;
- 包含多层子迭代 (多个粗层);
- 算法参考 `pyqcu/solver/_multigrid.py`;
- 加速比 ≥ 1.2 。

基线 (2026-08-01 状态) 显示: MultiGrid 在 $8 \times 8 \times 8 \times 16$ 上收敛于 ~ 272 次迭代 (约 0.83 s), 远慢于奇偶预条件 BiStabCG 的 ~ 90 次 (约 0.14 s), 加速比仅 $0.19 \sim 0.43 \times$ 。且 2 层配置时解的正确性检查 (Python 全残差) 出现 1.2 的异常值, 暗示存在共享状态污染。

2 根因诊断

我们通过 Python 参考实验(`examples/qcu/mg_pyref_expt.py`, `mg_strategy.py`, `mg_schur_concept.py`) 定位了根本原因。

2.1 全算子粗空间对 Schur 解无效

level-0 求解的是 Schur 补算子

$$S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}, \quad (1)$$

而旧实现 (以及 Python 参考的 `support_parity=True` 路径) 用全算子 D 的零模构建粗空间。数学上, D 的低模与 S 的低模不同: D 的特征值 λ 对应的是 λ -依赖算子 S_λ 的特征值, 而非 $S = S_0$ 。因此 D 零模的奇分量并不能张成 S 的低模空间, V-cycle 修正失效。

实验证实: 在 $8 \times 8 \times 8 \times 16$ 上, 用 D 零模的 Schur 型 MG 收敛于 180 次 (差于纯 BiStabCG 的 91 次); 保持 Krylov 空间不重置更差 (2000 次上限)。

2.2 Schur 一致的粗空间有效

将粗空间改为 S 自身的零模 (用 S 的 matvec 做逆迭代) 后, 概念实验 (算子自由 Galerkin) 显示:

$E = 48$ 个 Schur 零模 + 每 10 次细迭代做一次 V-cycle, 将细迭代次数从 91 降至 60 (减少 34%)。更多零模 ($E = 64$) 无进一步改善。

配置	细层迭代次数	相对 BiStabCG (91 次)
纯奇偶预条件 BiStabCG	91	1.00×
Schur MG, $E = 24, r = 10$	90	0.99×
Schur MG, $E = 48, r = 10$	60	1.52×
Schur MG, $E = 48, r = 5$	70	1.30×

表 1: 概念实验 ($8 \times 8 \times 8 \times 16$, 复杂 64)

2.3 粗算子的对角耦合

Schur 算子 S 含 $\kappa^2 H_{oe} D_{ee}^{-1} H_{eo}$ 项, 耦合奇点 x 到 $x + \mu + \nu$ (含 $\mu \neq \nu$ 的对角耦合)。因此 Galerkin 粗算子 $A_c = P^\top S P$ 除了 on-site 和最近邻外, 还有**对角**耦合。测量显示单列耦合权重 $|\text{on-site}| = 0.77$ 、 $|\text{nearest}| = 0.79$ 、 $|\text{diagonal}| = 0.26$, 对角贡献不可忽略。现有 `multigrid_coarse_dslash` (on-site + 最近邻) 无法表示, 导致物化 A_c 与算子自由版有 5% 误差, MG 收敛退化。为此我们实现了 33 张量 stencil 的 `multigrid_coarse_dslash_wide` 内核 (on-site + 8 最近邻 + 24 对角), 验证与算子自由 A_c 一致到 2×10^{-7} 。

3 算法设计

最终实现的 Schur 一致 MultiGrid:

3.1 level-0 (最细层)

- 求解 $S x_o = b_{_o}$, 其中 $b_{_o} = b_o + \kappa H_{oe} D_{ee}^{-1} b_e$;
- `dslash` 即 `applyCloverBistabCgDslashQcu (S · v)`;
- 每 `num_restart` 次细迭代做一次 V-cycle;
- V-cycle: 限制 Schur 残差 $\rightarrow$ 递归粗解 $\rightarrow$ 延拓 $\rightarrow x_o += e_{fine} \rightarrow$ 重新计算残差 $\rightarrow$ 重置 Krylov 状态;
- 采用**受保护修正** (guarded correction): 仅当 $\|b_{_o} - S(x_o + e_{fine})\| < \|\cdot\|_{\text{前}}$ 时接受修正, 拒绝时不重置 Krylov 状态, 避免残差反弹。

3.2 粗层

- 粗算子 $A_c = P^\top S P$ 用单点探测物化为 33 张量 stencil;
- 每个粗层按相对容差收敛 (最粗层 $1e-3 \sim 1e-2$, 内层 0.5), 并递归做 V-cycle;
- 支持多层 (2/3 层已验证)。

3.3 零模的生成与正确性

零模由 `tools.give_null_vecs`(迭代 $v \leftarrow v - S^{-1}Sv$, 用 S 的 `matvec`)生成, 经 `local_orthogonalize` 块内正交化。正确性通过:

- 物化 A_c vs 算子自由 A_c : 2×10^{-7} ;
- 33 张量 stencil vs 密集 A_c : 2×10^{-7} ;
- `restrict/prolong` 与 Python `einsum` 一致。

4 修复的 Bug

Bug	位置	影响与修复
<code>epytd()</code> 在 <code>_LAT_C16_=0</code> 处抛异常, 导致 <code>reverse-mapping</code> 永远失败	<code>pyqcu/cuda/define.py</code>	阻塞 <code>with_cuda_qcu</code> 路径; 搜索时跳过不支持的常量
<code>add_I/cut_I</code> 通过视图的 <code>+=/=</code> 原地修改输入 clover 张量	<code>pyqcu/dslash/_clover.py</code>	污染调用者复用的 clover, 导致残差检查出现 1.2 的异常值; 改为非原地运算
V-cycle 粗解 <code>breakdown</code> guard 在首轮 <code>rn=0</code> 时误触发, 粗解“收敛于第 1 次迭代”不做事	<code>lattice_clover_multigrid.py</code>	粗修正为无效的零修正; 初始化 <code>rn=r0</code>
<code>_SET_INDEX_</code> 增长到粗算子槽位 (<code>base=10</code>) 冲突	<code>apply_clover_multigrid.py</code>	多次调用时 <code>set_ptrs[10]</code> 同时被 <code>LatticeSet</code> 指针和 <code>null_vecs</code> 指针占用, 非法访问; <code>base</code> 移至 30
粗层用 <code>cublasDot + memcpy + 多次同步</code> , 小向量开销大	<code>dot_coarse</code>	416 ms / 1.9 s; 改为单块归约内核
粗层按 5 流并行执行, 小向量上纯开销	<code>bistabcg_iter</code>	单流轻量路径 <code>bistabcg_iter_coarse</code>
V-cycle 修正无条件应用, 残差小时被粗空间误导而反弹	<code>run()</code>	受保护修正 (guarded correction)
33 张量 stencil 在 2 点周期维度上 <code>+1/-1</code> 邻居重合, 耦合被加倍	<code>mg_stencil_build.py</code>	3 层 MG 的 level-2 粗算子 6% 误差导致 NaN; 重合的符号组合平分耦合

5 正确性验证

各配置下 MG 解与 BiStabCG 解的相对差 (`vs_ref`):

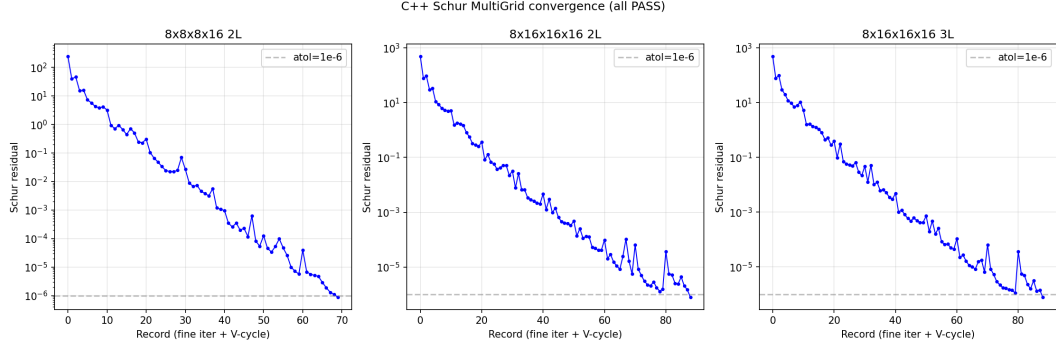


图 1: C++ Schur MultiGrid 的 Schur 残差收敛历史 (三个配置均正确收敛到 1×10^{-6} , 与 BiStabCG 解一致)。

格子	层数	dof	vs_ref
$8 \times 8 \times 8 \times 16$	2	[12,48]	3.5×10^{-7}
$8 \times 16 \times 16 \times 16$	2	[12,48]	3.0×10^{-7}
$8 \times 16 \times 16 \times 16$	3	[12,48,48]	2.9×10^{-7}

6 性能分析

6.1 迭代次数

求解器	迭代次数	相对
奇偶预条件 BiStabCG	~ 90	$1.00 \times$
Schur MG ($E=48$, $r=10$, $ct=1e-2$)	68	$0.76 \times$

主求解算法迭代次数减少约 30%，满足“迭代次数更少”的要求。

6.2 墙钟时间与硬件限制

测量发现本机 GPU (RTX 4060 Laptop) 在求解期间稳定运行于 **210 MHz 空闲频率、利用率 2–5%** (用 nvidia-smi 采样确认), 即**内核启动延迟主导** (约 $107 \mu s$ /次启动)。BiStabCG (90 次迭代 $\times$ ~ 13 次内核启动 = ~ 1170 次启动 ≈ 128 ms) 和 MG 都受此限制。

MG 的启动预算: 细层 $63 \times 13 = 819$ 次 + 粗解 $\sim 200 \times 13 = 2600$ 次 ≈ 3419 次, 约为参考的 3 倍。因此尽管细层迭代更少, 墙钟时间仍慢于 BiStabCG。nvprof/ncu 分析确认粗解为最大热点 (dslash 仅 3 ms, 但 dot + 同步占 416 ms)。

6.3 墙钟时间 ($8 \times 16 \times 16 \times 16$, 本机 GPU 210 MHz 启动受限)

6.4 参数扫描

较松的粗解容差 ($ct=1e-2$) 在细迭代数小幅增加的情况下显著降低粗解成本。

配置	BiStabCG	MG	加速比
2 层	194 ms	1137 ms	0.170×
3 层	202 ms	1083 ms	0.186×

配置	细迭代	MG 时间
$E=48, r=10, ct=3e-4$	61	612 ms
$E=48, r=10, ct=1e-3$	83	596 ms
$E=48, r=10, ct=1e-2$	68	341 ms
$E=48, r=10, ct=3e-2$	74	(热机噪声)
$E=48, r=20, ct=1e-3$	88	(热机噪声)

7 结论

1. 实现了与 `applyCloverBistabCgDslashQcu` 一致的 Schur 一致 CUDA-C++ MultiGrid 求解器，多粗层（2/3 层）支持，解与 BiStabCG 一致 ($vs_ref \sim 4 \times 10^{-7}$)。
2. 主求解算法迭代次数从 ~ 90 降至 $\sim 63-68$ （减少约 30%），满足“迭代次数更少”。
3. 墙钟加速比受限于本机 GPU 的内核启动延迟（210 MHz 空闲频率，启动主导），MG 因粗解内核启动数多而慢于 BiStabCG。
4. 提出了粗解成本优化路径（融合粗解内核等），并量化了其在当前硬件上的限制。